

# 基于Syslab求解优化问题

彭伟 王慧敏

苏州同元软控信息技术有限公司

2025年5月27日



TONGYUAN  
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

# 课程须知

➤ 本课程基于云化版本：

**Syslab 2025a构建**

➤ 本课程示例运行需要软件首选项中加载：

- 基础库
- 数学库
- 图形库
- 优化库
- 全局优化库

本课程所示示例均以附件形式提供，课程展示为伪代码

# CONTENTS

## 目录

### 导论

Syslab中的优化问题

### Part 1

基于问题的优化工作流程

### Part 2

基于求解器的问题优化流程

### Part 3

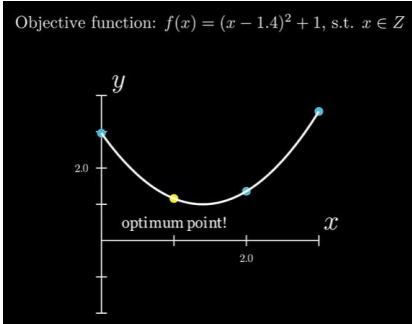
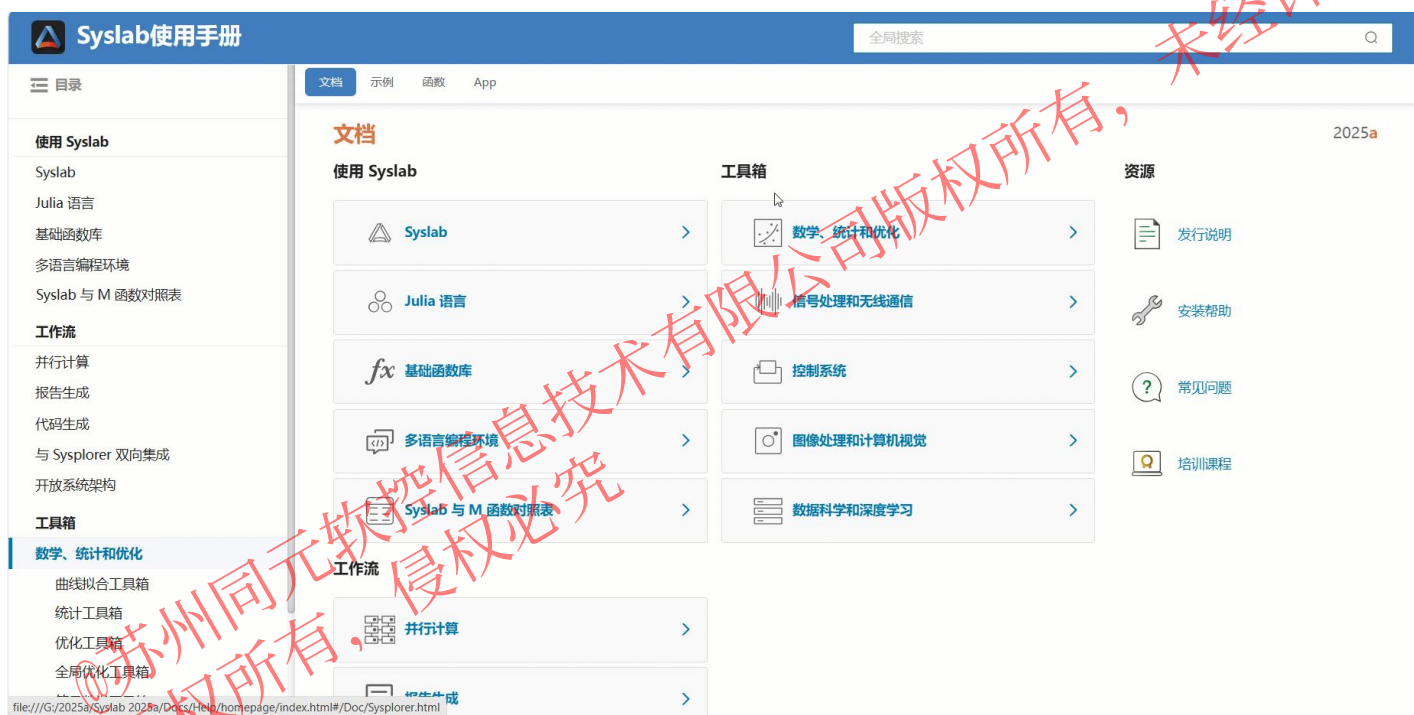
全局与多目标优化

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用  
版权所有，侵权必究

# Syslab中的优化问题

在生活或者工作中存在各种各样的最优化问题，比如每个企业和个人都要考虑的一个问题“在一定成本下，如何使利润最大化”等。最优化方法是一种数学方法，它是研究在给定约束之下如何寻求某些因素(的量)，以使某一(或某些)指标达到最优的一些学科的总称。

优化工具箱 提供了多个函数，这些函数可在满足约束的同时求出可最小化或最大化目标的参数



|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
|   | 2 |   |   | 3 |   | 4 |   |
| 6 |   |   |   |   |   |   | 3 |
|   |   | 4 |   |   | 5 |   |   |
|   |   |   | 8 |   | 6 |   |   |
| 8 |   |   |   | 1 |   |   | 6 |
|   |   |   | 7 |   | 5 |   |   |
|   | 7 |   |   |   | 6 |   |   |
| 4 |   |   |   |   |   |   | 8 |
|   | 3 |   |   | 4 |   | 2 |   |



# 基于问题的优化工作流程

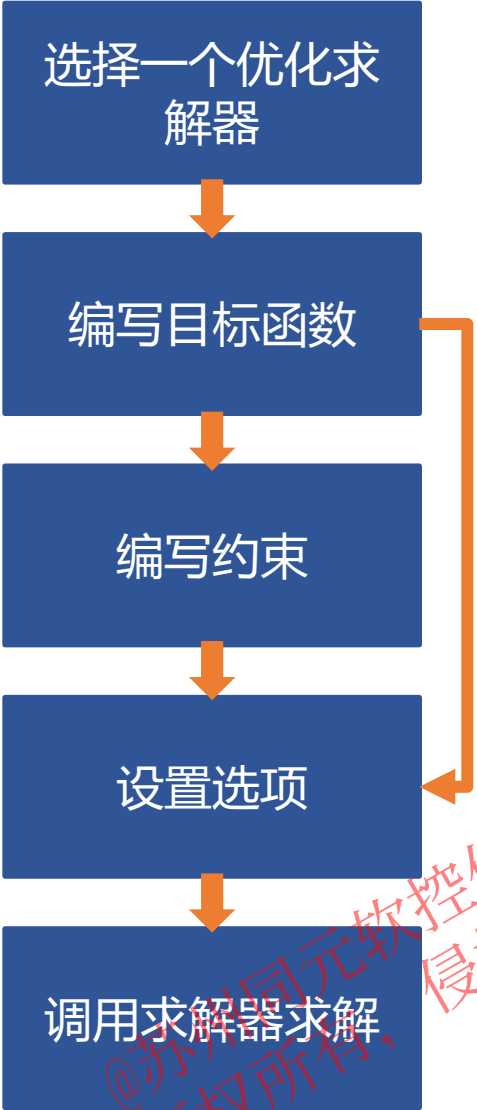
## 基于问题的优化工作流程



| 函数           | 描述                                  | 示例                                                                                                       |
|--------------|-------------------------------------|----------------------------------------------------------------------------------------------------------|
| optimproblem | 创建一个默认属性的优化问题                       | prob = optimproblem()                                                                                    |
| optimvar     | 创建优化变量，可指定变量类型和上下界                  | x = optimvar("x")<br>x = optimvar("x",3)<br>x = optimvar("x",3,3,3,Type="integer",Lowerbnd=0,Upperbnd=1) |
| @fcn2expr    | 将目标函数转换为表达式                         | prob.Objective = @fcn2expr gamma(x) x                                                                    |
| Solve        | 求解优化问题                              | sol, = Solve(prob)                                                                                       |
| evaluate     | 优化表达式在某点处的数值，或求优化问题中目标和约束表达式在一组点处的值 | xmat = [3 -1;0 1;2 6]<br>sol = (x = xmat, y = [4 -3]) val = evaluate(expr,sol)                           |

# 基于求解器的优化工作流程

## 基于求解器的优化工作流程



| 目标类型     | 求解器                                                                               |
|----------|-----------------------------------------------------------------------------------|
| 非线性优化    | fmincon<br>fminunc<br>fminbnd<br>fminsearch<br>fseminf<br>fminimax<br>fgoalattain |
| 非线性最小二乘  | lsqcurvefit<br>lsqnonlin                                                          |
| 非线性方程求解  | fsolve<br>fzero                                                                   |
| 多目标      | fminimax                                                                          |
| 线性规划     | linprog                                                                           |
| 混合整数线性规划 | intlinprog                                                                        |
| 线性最小二乘   | lsqlin<br>lsqnonnegm                                                              |
| 二次规划     | quadprog                                                                          |

| 约束类型    | 描述                                                                            |
|---------|-------------------------------------------------------------------------------|
| 边界约束    | 单个分量的下界和上界; $x \geq l$ 和 $x \leq u$ 。                                         |
| 线性不等式约束 | $A \cdot x \leq b$ 。A 是 $m \times n$ 矩阵, 它表示 $n$ 维向量 $x$ 的 $m$ 个约束。b 为 $m$ 维。 |
| 线性等式约束  | $A_{eq} \cdot x = b_{eq}$ 。等式约束与不等式约束具有相同的形式。                                 |
| 非线性约束   | $c(x) \leq 0$ 且 $ceq(x) = 0$ 。c 和 ceq 都是表示几个约束的标量或向量。                         |

| 函数名          | 简介                  |
|--------------|---------------------|
| optimoptions | 创建优化选项              |
| optimset     | 创建或修改优化 options 结构体 |
| optimget     | 优化选项值               |
| resetoptions | 重置选项                |

# Part1-基于问题的 workflows



@苏州同元软控信息技术有限公司版权所有，  
侵权必究

# 1. 基于问题的优化-以Syslab求解数独问题为例

谜题是用 1 到 9 的整数填充  $9 \times 9$  网格，要求这九个数字中的每个数字在每一行、每一列和每个九宫格 ( $3 \times 3$  粗线正方形) 中都只出现一次，不能重复。网格已根据提示进行了部分填充，任务是填充网格的其余部分。

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
|   | 2 |   |   | 3 |   |   | 4 |   |
| 6 |   |   |   |   |   |   |   | 3 |
|   |   | 4 |   |   |   | 5 |   |   |
|   |   |   | 8 |   | 6 |   |   |   |
| 8 |   |   |   | 1 |   |   |   | 6 |
|   |   |   | 7 |   | 5 |   |   |   |
|   |   | 7 |   |   |   | 6 |   |   |
| 4 |   |   |   |   |   |   |   | 8 |
|   | 3 |   |   | 4 |   | 2 |   |   |

### 初始谜题

这是一个带有提示的数据矩阵 B。第一行 B(1,2,2) 表示第 1 行第 2 列的整数提示为 2。第二行 B(1,5,3) 表示第 1 行第 5 列的整数提示为 3。以下是整个矩阵 B。

关键思路是将谜题从  $9 \times 9$  正方形网格转换为一个由二元值 (0 或 1) 组成的  $9 \times 9 \times 9$  立方数组。将这个立方数组想象成由 9 个正方形网络上下堆叠在一起，其中每一层对应于 1 到 9 中的一个整数。顶部网格是数组的第一个正方形层，只要解或提示包含整数 1，则该层对应位置就为 1。只要解或提示包含整数 2，第二层对应位置就为 1。只要解或提示包含整数 9，第九层对应位置就为 1。

这种表示非常适合二元整数规划。

此处不需要目标函数，目标函数也可能是常数项 0。问题实际上只是求可行解，也就是满足所有约束的解。但是，为在整数规划求解器内部打破平衡，同时保证求解速度，这里使用一个非常数目标函数。

B = [1 2 2;  
1 5 3;  
1 8 4;  
2 1 6;  
2 9 3;  
3 3 4;  
3 7 5;  
4 4 8;  
4 6 6;  
5 1 8;  
5 5 1;  
5 9 6;  
6 4 7;  
6 6 5;  
7 3 7;  
7 7 6;  
8 1 4;  
8 9 8;  
9 2 3;  
9 5 4;  
9 8 2];



# 1. 基于问题的优化-以Syslab求解数独问题为例

假设解  $x$  用一个  $9 \times 9 \times 9$  的二元数组表示。那  $x$  应具备哪些特性呢？首先，由于二维网格  $(i,j)$  的每个方格中都有且仅有一个值，因此三维数组项  $x(i,j,1), \dots, x(i,j,9)$  中有且仅有一个非零元素。换句话说，对于每个  $i$  和  $j$ ，都满足

$$\sum_{k=1}^9 x(i, j, k) = 1.$$

在二维网格的每一行  $i$  中，1 到 9 中的每个数字都只能出现一次。换句话说，对于每个  $i$  和  $k$ ，都满足

$$\sum_{j=1}^9 x(i, j, k) = 1.$$

二维网格中的每一列  $j$  也具有同样的特性：对于每个  $j$  和  $k$ ，都满足

$$\sum_{i=1}^9 x(i, j, k) = 1.$$

$3 \times 3$  粗线网格具有类似的约束。对于  $1 \leq i \leq 3$  且  $1 \leq j \leq 3$  的网格元素，对于每层  $1 \leq k \leq 9$  都满足

$$\sum_{i=1}^3 \sum_{j=1}^3 x(i, j, k) = 1.$$

要表示全部九个粗线网格，只需将每个  $i$  和  $j$  索引加上 3 或 6：

$$\sum_{i=1}^3 \sum_{j=1}^3 x(i+U, j+V, k) = 1, \text{ 其中 } U, V \in \{0, 3, 6\}.$$

## 将提示表示为约束

每个初始值（提示）都可以表示为一个约束。假设在  $1 \leq m \leq 9$  时， $(i,j)$  格点处的提示为  $m$ 。则  $x(i,j,m)=1$ 。约束

$$\sum_{k=1}^9 x(i, j, k) = 1$$

能确保在  $k \neq m$  时，所有其他  $x(i,j,k)=0$ 。

# 1. 基于问题的优化-以Syslab求解数独问题为例

根据基于问题的优化工作流程，构建优化问题

```
using TyOptimization
sudpuzzle = optimproblem()
#创建一个优化变量 x, 它是一个大小为 9×9×9 的二元变量。
x = optimvar("x", 9, 9, 9, Type="integer", Lowerbnd=0,
Upperbnd=1)
#创建一个具有任意目标函数的优化问题
mul = ones(1, 1, 9);
mul = cumsum(mul, dims=3);
sudpuzzle.Objective = @fcn2expr sum(sum(sum(x,
dims=1), dims=2) .* mul) x;
#sudpuzzle.Objective =@fcn2expr x
#将约束表示为：在每个坐标方向上 x 的总和为 1,这里需要注意,
单个约束, 左右两侧的维度需要一致
cons1 = sum(x, dims=1) == ones(1, 9, 9);
cons2 = sum(x, dims=2) == ones(9, 1, 9);
cons3 = sum(x, dims=3) == ones(9, 9, 1);
```

#还要创建粗线网格总和等于 1 的约束。

#按照案例

```
majorg = optimconstr(==, 3, 3);
```

#约束赋值不支持使用向量式语句

```
for u = 1:3
```

```
    for v = 1:3
```

```
        arr = x[3*(u-1)+1:3*(u-1)+3, 3*(v-1)+1:3*(v-1)+3, :]
```

```
        majorg[u, v] = sum(sum(arr, dims=1), dims=2) ==
```

```
ones(1, 1, 9)
```

```
    end
```

```
end
```

```
show(majorg)
```

```
sudpuzzle.Constraints = (cons1=cons1, cons2=cons2,
```

```
cons3=cons3, majorg=majorg);
```

#添加约束方式与MATLAB不同, 这里需要一次性添加所有约束

#在包含提示的格点处设置下界 1 来包括初始提示值。此设置将对应项的值固定为 1, 从而将每个提示数处的解设置为提示数本身。

```
for u = 1:size(B::Matrix{Int64}, 1)
```

```
    x.LowerBound[B[u, 1], B[u, 2], B[u, 3]] = 1
```

```
end
```

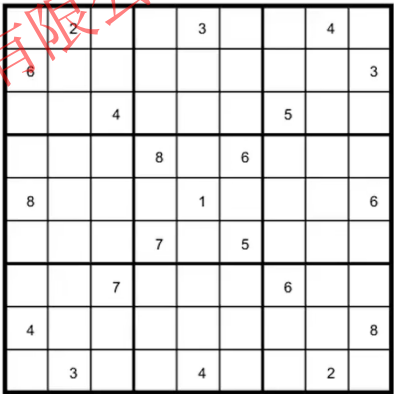
#检查问题

```
show(sudpuzzle)
```

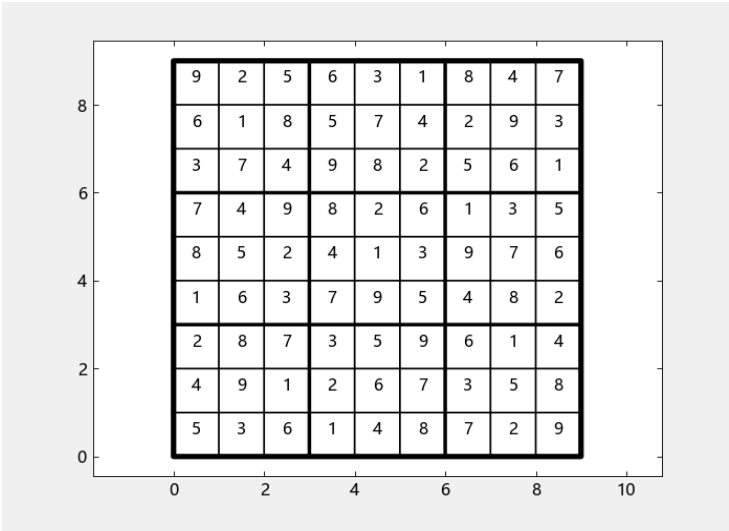
# 1. 基于问题的优化-以Syslab求解数独问题为例

```
#求解数独谜题
sudsoln, = Solve(sudpuzzle)

#显示求解结果
y = ones(size(sudsoln.x));
for k = 2:9
    y[:, :, k] .= k
end
include("drawsudoku.jl")
S = sudsoln.x .* y;
S = sum(S, dims=3);
S = squeeze(S) #删除长度为 1 的维度
drawSudoku(Int.(S))
```



Solving problem using intlinprog.  
Running HiGHS 1.6.0: Copyright (c) 2023 HiGHS  
under MIT licence terms  
Presolving model  
221 rows, 246 cols, 990 nonzeros  
45 rows, 30 cols, 200 nonzeros  
23 rows, 21 cols, 92 nonzeros  
17 rows, 16 cols, 76 nonzeros  
16 rows, 16 cols, 72 nonzeros  
Objective function is integral with scale 1



## Part2-基于求解器的优化问题

1. 线性规划问题
2. 二次规划问题
3. 整数规划问题
4. 非线性规划问题

@苏州同元软控信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

## 1.1 线性规划问题

**线性规划 (Linear programming, 简称LP) 是运筹学中研究较早、发展较快、应用广泛、方法较成熟的一个重要分支, 它是辅助人们进行科学管理的一种数学方法。**研究线性约束条件下线性目标函数的极值问题的数学理论和方法。

- 线性规划广泛应用于军事作战、经济分析、经营管理和工程技术等方面。为合理地利用有限的人力、物力、财力等资源作出的最优决策, 提供科学的依据。
- 从实际问题中建立数学模型一般有以下三个步骤:
  - 根据影响所要达到目的的因素找到决策变量;
  - 由决策变量和所在达到目的之间的函数关系确定目标函数;
  - 由决策变量所受的限制条件确定决策变量所要满足的约束条件。

# 1.2 线性规划的数学模型

- 线性规划问题的目标函数及约束条件均为线性函数；
- 约束条件记为 *s.t.*(即 subject to)。目标函数可以是求最大值，也可以是求最小值，约束条件的不等号可以是<也可以是>。

一般线性规划问题的（数学）标准型为

$$\begin{aligned} &\min(\max) f^T X \\ &s.t. \begin{cases} AX \leq b \\ AeqX = beq \\ lb \leq X \leq ub \end{cases} \end{aligned}$$

其中 $X$ 为 $n$ 维未知向量， $f^T$  为目标函数系数向量；  
小于等于约束系数矩阵 $A$ 为 $m \times n$ 矩阵，  
 $b$ 为其右端 $m$ 维列向量，  
 $Aeq$ 为等式约束系数矩阵，  
 $beq$ 为等式约束右端常数列向量；  
 $lb$ 、 $ub$ 为自变量取值上界与下界约束的 $n$ 维常数向量。

在Syslab中需要将此类优化问题转成求最小值



## 1.3 线性规划的求解格式

线性规划问题求最优解函数，调用格式：

```
x,fval,exitflag,output,lambda =linprog(f, A, b, Aeq, beq, lb, ub, options)
```

求解  $\min f'x$ ，满足  $Ax \leq b$ 。该函数返回值为：

- $x$  - 线性规划的解
- $fval$  - 目标函数在解  $x$  处的值：  $fval = f'x$
- $exitflag$  - 说明退出条件的值：正值表示目标函数收敛于解 $x$ 处；负值不收敛，零值表示已达到函数评价或迭代的最大次数
- $output$  - 包含优化过程信息的结构体：如iterations、algorithm、constrviolation、firstorderopt等
- $lambda$  - 在解  $x$  处的拉格朗日乘数



# 1.4.1 线性规划例题解析1

**例题1（生产安排）：**假设某厂计划生产甲、乙、丙、丁四种产品，现库存主要材料有A类3600公斤、B类2900公斤、C类3000公斤、D类2800公斤、E类2200公斤。每件产品需用材料情况如下所示。甲单位产品的利润75元、乙利润120元、丙利润90元、丁利润105元，且由于市场需求，每种产品最少需要生产50件。

（注：可考虑用料最少或考虑用料费用，转化为多目标规划问题）

问：如何安排生产，才能使该厂所获的利润最大？

| 产品/材料 |   | 产品 |    |   |    | 库存材料 |
|-------|---|----|----|---|----|------|
|       |   | 甲  | 乙  | 丙 | 丁  |      |
| 主要材料  | A | 9  | 4  | 7 | 5  | 3600 |
|       | B | 4  | 5  | 6 | 10 | 2900 |
|       | C | 5  | 10 | 8 | 5  | 3000 |
|       | D | 3  | 8  | 9 | 7  | 2800 |
|       | E | 7  | 6  | 4 | 8  | 2200 |





# 1.4.1 线性规划例题解析1

$$\max f = 75x_1 + 120x_2 + 90x_{3+} + 105x_4$$

$$s.t. \begin{cases} 9x_1 + 4x_2 + 7x_3 + 5x_4 \leq 3600 \\ 4x_1 + 5x_2 + 6x_3 + 10x_4 \leq 2900 \\ 5x_1 + 10x_2 + 8x_3 + 5x_4 \leq 3000 \\ 3x_1 + 8x_2 + 9x_3 + 7x_4 \leq 2800 \\ 7x_1 + 6x_2 + 4x_3 + 8x_4 \leq 2200 \\ x_i \geq 50, i = 1, 2, 3, 4 \end{cases}$$

```
using TyOptimization
fcoeff = -[75, 120, 90, 105]; #目标函数系数向量
A = [9 4 7 5;4 5 6 10;5 10 8 5;3 8 9 7;7 6 4 8]; #约束不等式系数矩阵
b = [3600, 2900, 3000, 2800, 2200]; #约束不等式右端向量
Aeq = []; #约束等式系数矩阵
beq = []; #约束等式右端向量
lb = 50*ones(4); #决策变量下限
ub = []; #决策变量上限
options = optimoptions(:linprog,Algorithm="dual-simplex"); #设置选项
x,fval,exitflag,output= linprog(fcoeff,A,b,Aeq,beq,lb,ub,options)
fval = - fval
```

生产安排方案为：甲产品生产50件，乙产品生产161件，丙产品生产107件，丁产品生产57件，最大利润为38679元。



# 1.4.2 线性规划例题解析2

**例2（运输问题）：**有A、B、C、D四个食品加工厂，负责供给甲乙丙丁戊五个市场。四个厂每天生产食品箱数上限、五个市场每天的需求量和从各厂到各市场运输费（元/每箱）如表所示：

| 收点/发点 |   | 市场 |    |    |    |    | 生产数 |
|-------|---|----|----|----|----|----|-----|
|       |   | 甲  | 乙  | 丙  | 丁  | 戊  |     |
| 工厂    | A | 2  | 1  | 3  | 2  | 1  | 60  |
|       | B | 1  | 3  | 2  | 1  | 3  | 40  |
|       | C | 3  | 4  | 1  | 1  | 2  | 50  |
|       | D | 2  | 1  | 3  | 2  | 2  | 55  |
| 需求量   |   | 20 | 35 | 22 | 34 | 30 |     |

问题：求在基本满足供需平衡的约束条件下使总运输费用小。

# 1.4.2 线性规划例题解析2

解：设 $a_{ij}$ 为由工厂*i*运到市场*j*的费用， $x_{ij}$ 是由工厂*i*运到市场*j*的箱数。 $b_i$ 是工厂*i*的生产量， $d_j$ 是市场*j*的需求量。

$$A = \begin{bmatrix} 2 & 1 & 3 & 2 & 1 \\ 1 & 3 & 2 & 1 & 3 \\ 3 & 4 & 1 & 1 & 2 \\ 2 & 1 & 3 & 2 & 2 \end{bmatrix}$$

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} & x_{15} \\ x_{21} & x_{22} & x_{23} & x_{24} & x_{25} \\ x_{31} & x_{32} & x_{33} & x_{34} & x_{35} \\ x_{41} & x_{42} & x_{43} & x_{44} & x_{45} \end{bmatrix}$$

$$b = [60, 40, 50, 55]$$

$$d = [20, 35, 33, 34, 30]$$

$$\begin{aligned} \min f &= \sum_{i=1}^4 \sum_{j=1}^5 a_{ij} \cdot x_{ij} \\ s.t. &\begin{cases} \sum_{j=1}^5 x_{ij} \leq b_i, & i = 1, 2, 3, 4 \\ \sum_{i=1}^4 x_{ij} = d_j, & j = 1, 2, 3, 4, 5 \\ x_{ij} \geq 0 \end{cases} \end{aligned}$$



# 1.4.2 线性规划例题解析2

```
using TyOptimization
fcoeff = [2, 1, 3, 2, 1, 3, 4, 1, 3, 2, 1, 3, 2, 1, 1,
2, 1, 3, 2, 2];#运输费用的向量的形式
#线性约束的不等式矩阵,可以使用repeat函数, 对数组进行复制
A = [1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0;
0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0;
0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0;
0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1];
b = [60, 40, 50, 55];
#线性约束的等式矩阵
Aeq = [1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0;
0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0;
0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0;
0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0;
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1];
beq = [20, 35, 33, 34, 30]
lb = zeros(20)#决策变量下限
ub = []; #决策变量上限
```

```
#单纯形法
options = optimoptions(:linprog,
Algorithm="dual-simplex");#设置选项
x, fval, exitflag, output = linprog(fcoeff, A, b,
Aeq, beq, lb, ub, options)
x=reshape(x,4,5)
```

使用内点法求解的结果不一样，可做一下对比

```
#内点法
options = optimoptions(:linprog,
Algorithm="interior-point-legacy");#设置选项
x, fval, exitflag, output = linprog(fcoeff, A, b,
Aeq, beq, lb, ub, options)
x=reshape(x,4,5)
```

## 二次规划问题

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用  
侵权必究

# 2.1 二次规划

**二次规划**是非线性中的一类特殊数学问题，在很多方面都有应用，如投资组合和序列二次规划等。如果某非线性规划的目标函数为自变量的二次函数，约束条件全是线性函数，就称这样规划为**二次规划**。其数学模型为：

二次规划的标准型

$$\begin{aligned} \min Z &= \frac{1}{2} X^T H X + f^T X \\ s.t. &\begin{cases} A X \leq b \\ Aeq X = beq \\ lb \leq X \leq ub \end{cases} \end{aligned}$$

其中 $X$ 为 $n$ 维未知向量， $f$ 为目标函数系数向量；  
 $H$ 为Hessian矩阵，  
小于等于约束系数矩阵 $A$ 为 $m \times n$ 矩阵，  
 $b$ 为其右端 $m$ 维列向量，  
 $Aeq$ 为等式约束系数矩阵，  
 $beq$ 为等式约束右端常数列向量；  
 $lb$ 、 $ub$ 为自变量取值上界与下界约束的 $n$ 维常数向量。

如果Hessian矩阵是半正定的，则该式是一个凸二次规划，在这种情况下该问题的困难程度类似于线性规划。如果有至少一个向量满足约束并且在可行域有下界，则凸二次规划问题就有一个全局最小值。  
如果是正定的，则这类二次规划为严格的凸二次规划，那么全局最小值就是唯一的。  
如果是一个非正定矩阵，则为非凸二次规划，这类二次规划更有挑战性，因为它们有多个平稳点和局部极小值点。



## 2.2 二次规划的求解格式

二次线性规划问题求最优解函数，调用格式：

```
x,fval,exitflag,output,lambda =quadprog(H,f,A,b,Aeq,beq,lb,ub,x0,options)
```

- x - 线性规划的解
- fval - 目标函数在解 x 处的值： $fval = 0.5 * x' * H * x + f' * x$
- exitflag - 说明退出条件的值：正值表示目标函数收敛于解x处；负值不收敛，零值表示已达到函数评价或迭代的最大次数
- output - 包含优化过程信息的结构体：如iterations、algorithm、constrviolation、firstorderopt等
- lambda - 在解 x 处的拉格朗日乘数



## 2.3 二次规划例题解析1

$$\min f(x) = \frac{1}{2}x_1^2 + x_2^2 - x_1x_2 - 2x_1 - 6x_2$$

$$s.t. \begin{cases} x_1 + x_2 \leq 2 \\ -x_1 + 2x_2 \leq 2 \\ 2x_1 + x_2 \leq 3 \\ x_1, x_2 \geq 0 \end{cases}$$



$$\min Z = \frac{1}{2}X^T HX + f^T X$$
$$s.t. \begin{cases} A X \leq b \\ Aeq X = beq \\ lb \leq X \leq ub \end{cases}$$

$$X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad f = \begin{bmatrix} -2 \\ -6 \end{bmatrix} \quad H = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$$

$$A = \begin{bmatrix} 1 & 1 \\ -1 & 2 \\ 2 & 1 \end{bmatrix} \quad b = \begin{bmatrix} 2 \\ 2 \\ 3 \end{bmatrix} \quad lb = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

```
using TyOptimization
H = [1 -1; -1 2]
f = [-2, -6]
A = [1 1; -1 2; 2 1]
b = [2, 2, 3]
lb=[0,0]

#options =

optimoptions(:quadprog,Algorithm="active-set")

x,fval,exitflag,output,lambda =

quadprog(H,f,A,b,[],[],lb,[],[])
```

## 整数规划问题



@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用  
版权所有，侵权必究

# 3.1 整数规划数学模型

数学规划中的变量（部分或全部）限制为整数时，称为整数规划。

对于整数线性规划模型大致可分为两类

- (1) 变量全限制为整数时，称纯（完全）整数规划。
- (2) 变量部分限制为整数的，称混合整数规划。

$$\begin{aligned} \min_x \quad & z = f^T x \\ \text{s.t.} \quad & \begin{cases} x(\text{intcon}) \text{ are integers} \\ A \cdot x \leq b \\ Aeq \cdot x = beq \\ lb \leq x \leq ub \end{cases} \end{aligned}$$

我们现在利用Syslab的intlinprog函数来实现整数规划或混合整数规划的求解。函数intlinprog的使用格式是：

```
x,fval = intlinprog(f,intcon,A,b)
x,fval = intlinprog(f,intcon,A,b,Aeq,beq)
x,fval = intlinprog(f,intcon,A,b,Aeq,beq,lb,ub)
x,fval = intlinprog(f,intcon,A,b,Aeq,beq,lb,ub,x0)
```

与前面的linprog相比，主要多了一个intcon也就是整数约束，告诉系统那几个变量是整数变量，intcon 的值在 1 到 length(f) 范围内，如intcon = [1,2,7] 表示 x[1]、x[2] 和 x[7]仅取整数值。

## 3.2 整数规划例题解析1

**例：**某快餐店一周中每天需要不同数目的雇员，设周一至少 $a_1$ 人，周二至少 $a_2$ 人，周三至少 $a_3$ 人，周四至少 $a_4$ 人，周五至少 $a_5$ 人，周六至少 $a_6$ 人，周日至少 $a_7$ 人，又规定雇员需连续工作5天，每人每天的工资为 $C$ 元。问快餐店怎样聘用雇员才能满足需求，又能使总聘用费用最少？

提示：由于每个雇员需连续工作5天，故快餐店聘用的总人数不一定是每天聘用人数之和。我们定义周一开始工作的雇员数为 $x_1$ ，周日开始工作的雇员数为 $x_7$ ，则一周的聘用总费用为： $z = C(x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7)$ ，由于除了周二和周三开始工作的雇员之外，其余的雇员都会在周一工作，所以周一至少应有 $a_1$ 人的约束应表示为：

$$x_1 + x_4 + x_5 + x_6 + x_7 \geq a_1$$

类似地可以得出其它的约束条件。

现给定 $C = 100$ 元， $a_1 = 16$ 人， $a_2 = 15$ 人， $a_3 = 16$ 人， $a_4 = 19$ 人， $a_5 = 14$ 人， $a_6 = 12$ 人， $a_7 = 18$ 人，请给出问题的数学模型，并用Syslab来求解。

## 3.2 整数规划例题解析1

解：数学模型为：

$$\begin{aligned} \min z &= 100(x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7) \\ \text{s. t. } &\begin{cases} x_1 + x_4 + x_5 + x_6 + x_7 \geq 16 \\ x_2 + x_5 + x_6 + x_7 + x_1 \geq 15 \\ x_3 + x_6 + x_7 + x_1 + x_2 \geq 16 \\ x_4 + x_7 + x_1 + x_2 + x_3 \geq 19 \\ x_5 + x_1 + x_2 + x_3 + x_4 \geq 14 \\ x_6 + x_2 + x_3 + x_4 + x_5 \geq 12 \\ x_7 + x_3 + x_4 + x_5 + x_6 \geq 18 \\ x_i, i = 1, 2, 3, 4, 5, 6, 7 \text{ 为整数} \end{cases} \end{aligned}$$

是一个整数规划模型，注意实现的时候要将约束条件的不等号反向，因此，实现的程序为：

```
using TyOptimization
f=100*[1, 1, 1, 1, 1, 1, 1]
A=-[1 0 0 1 1 1 1; 1 1 0 0 1 1 1; 1 1 1 0 0 1
1;1 1 1 1 0 0 1; 1 1 1 1 1 0 0; 0 1 1 1 1 1
0; 0 0 1 1 1 1 1]
b=-[16, 15, 16, 19, 14, 12, 18]
intcon=[1,2, 3, 4, 5, 6, 7] # 表示7个变量都是整数
x,fval=intlinprog(f,intcon,A,b)
#运行结果为:
x,fval = ([2.0, 2.0, 4.0, 3.0, 3.0, 0.0, 8.0],
2200.0)
```

表示周一到周日开始工作的雇员数分别为2, 2, 4, 3, 3, 0, 8, 总聘用费用为2200元。



## 非线性优化问题



@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用  
版权所有，侵权必究

## 4.1 非线性优化问题-非线性一元函数的最小值

非线性优化包括相当丰富的内容，我们这里就提供的一些函数来介绍相关函数的用法及其所能解决的问题。

### 1.非线性一元函数的最小值

MWORKS命令为fminbnd()，其使用格式为：

```
X,fval,exitflag,output = fminbnd(fun,x1,x2)
```

```
X,fval,exitflag,output = fminbnd(fun,x1,x2,options)
```

其中：fun为目标函数，x1，x2为变量的边界约束，即 $x_1 \leq x \leq x_2$ ，x为返回的满足fun取得最小值的x的值，而fval则为此时的目标函数值。exitflag>0表示计算收敛，exitflag=0表示超过了最大的迭代次数，exitflag<0表示计算不收敛，返回值output有3个分量，其中iterations是优化过程中迭代次数，funcCount是代入函数值的次数，algorithm是优化所采用的算法。



## 4.1 非线性优化问题-非线性一元函数的最小值

例：求函数 $f(x) = \frac{x^5+x^3+x^2-1}{e^{x^2}+\sin(-x)}$ 在区间 $[-2,2]$ 的最小值和相应的 $x$ 值。

解决此问题的程序为：

```
using TyOptimization
function f1(x)
    y = (x^5 + x^3 + x^2 - 1) / (exp(x^2) + sin(-x))
    return y
end
X, fval, exitflag, output = fminbnd(f1, -2, 2)
```

(0.21759143139606002, -1.1312238928807807, 1, iterations: 12

funcCount: 13

algorithm: golden section search, parabolic interpolation

message: Optimization terminated:

the current x satisfies the termination criteria TolX of 0.0001)

## 4.2 非线性优化问题-有约束非线性多变量优化问题

### 有约束非线性多变量优化问题

由线性规划我们看到优化要处理各种约束条件，在非线性规划中问题就更加复杂，除了线性规划中的那些约束外，还要增加非线性约束。函数fmincon()可以处理有约束的非线性多元函数的优化问题。

有约束多变量优化问题的数学模型为：求一组变量 $x_1, x_2, \dots, x_n$ ，满足在给定的约束条件下，使目标函数 $f(x_1, x_2, \dots, x_n)$ 最小。目标函数一般为非线性函数，约束条件分为线性不等式约束、线性等式约束、变量边界约束和非线性约束几部分。除非线性约束外，表示方法与线性规划相同。函数fmincon()的具体格式为：

```
x,fval,exitflag,output,lambda,grad,hessian = fmincon(fun,x0,A,b)
```

```
x,fval,exitflag,output,lambda,grad,hessian = fmincon(fun,x0,A,b,Aeq,beq)
```

```
x,fval,exitflag,output,lambda,grad,hessian = fmincon(fun,x0,A,b,Aeq,beq,lb,ub)
```

```
x,fval,exitflag,output,lambda,grad,hessian = fmincon(fun,x0,A,b,Aeq,beq,lb,ub,nonlcon)
```

```
x,fval,exitflag,output,lambda,grad,hessian =
```

```
fmincon(fun,x0,A,b,Aeq,beq,lb,ub,nonlcon,options)
```

## 4.2 非线性优化问题-有约束非线性多变量优化问题

### 有约束非线性多变量优化问题

用于求解以下问题的最小值:

$$\begin{aligned} \min z &= fun(x) \\ s.t. \begin{cases} c(x) \leq 0 \\ ceq(x) = 0 \\ A \cdot x \leq b \\ Aeq \cdot x = beq \\ lb \leq x \leq ub \end{cases} \end{aligned}$$

b 和 beq 是向量, A 和 Aeq 是矩阵, A和b表示线性不等式约束, Aeq, Beq表示线性等式约束, c(x) 和 ceq(x) 是返回向量的函数, fun(x) 是返回标量的函数。f(x)、c(x) 和 ceq(x) 可以是非线性函数。x、lb 和 ub 作为向量传递。x0为变量的初始值, lb和ub分别为变量的下界和上界约束, nonlcon表示非线性约束条件, options为控制优化过程的优化参数向量。

options = optimoptions(:fmincon) 用于为 fmincon 求解器创建默认选项。  
options = optimoptions(:fmincon,Algorithm="sqp",MaxIterations=1500)为 fmincon 设置选项, 以使用 sqp 算法和最多 1500 次迭代。

参数中fun为目标函数, x为返回的满足要求的变量的值, 返回值fval为目标函数。exitflag>0表示优化结果收敛于解, exitflag=0表示优化超过了函数值的计算次数, exitflag<0表示优化不收敛。lambda是拉格朗日乘子, 显示那个约束条件有效。grad表示梯度, hessian表示汉森矩阵。



## 4.2 非线性优化问题-有约束非线性多变量优化问题

求 $[x_1, x_2]$ , 使得目标函数 $f(x_1, x_2) = e^{x_1}(4x_1^2 + 2x_2^2 + 4x_1x_2 + 2x_2 + 1)$ 在约束条件 $1.5 + x_1 * x_2 - x_1 - x_2 \leq 0$ ,  $-x_1 * x_2 \leq 10$ 下取得最小值。

```
using TyOptimization
objfun = x -> exp(x[1]) * (4 * x[1]^2 + 2 *
x[2]^2 + 4 * x[1] * x[2] + 2 * x[2] + 1);
# 约束条件的函数为:
mycon = x -> begin
    c = [1.5 + x[1] * x[2] - x[1] - x[2]; -x[1]
* x[2] - 10]
    ceq = []
    return c, ceq
end
x0=[-1; 1];
options =
optimoptions(:fmincon, Algorithm="sqp")
x,fval,exitflag,output=fmincon(objfun,x0,[],[],
,[],[],[],[],mycon,options)
```

```
([-9.547386058778851, 1.0474060587790412],
0.02355072618943104, 1, (iterations = 10, funcCount = 38,
algorithm = "sqp", constrviolation = 0.0, stepsize =
8.56139067436345e-5, lssteplength = 1.0, firstorderopt =
7.95920938517769e-7), (eqlin = Float64[], eqnonlin =
Float64[], ineqlin = Float64[], ineqnonlin =
[0.01635281872512767, 0.018304977441471262], lower =
Float64[], upper = Float64[]), [0.018397290980284288, -
0.0022843983994440156, 0.0, 0.0, 0.0], [0.015945683598002112
-0.00560204270947437; -0.005602042709474411
0.013979448325346753])
```

## Part3-全局与多目标优化



@苏州同元软控信息技术有限公司版权所有，  
侵权必究

### 3.1 全局优化

全局优化工具箱包含遗传算法、粒子群优化、模拟退火、直接搜索、多起点搜索、替代优化和多目标优化。您可以将这些求解器用于目标或约束函数是连续的、不连续的、随机的、无导数的、或包括模拟或黑盒函数的优化问题。

| 分类       | 函数名称                | 描述                      |
|----------|---------------------|-------------------------|
| 全局或多起点搜索 | createOptimProblem  | 创建优化问题结构                |
|          | list                | 列出起点                    |
|          | run                 | 运行多启动求解器                |
|          | GlobalSearch        | 寻找全局最小值                 |
|          | MultiStart          | 寻找多个局部最小值               |
|          | CustomStartPointSet | 自定义起点                   |
|          | GlobalOptimSolution | 优化方案                    |
|          | RandomStartPointSet | 随机起点                    |
|          | packfcn             | 结合目标和非线性约束函数            |
| 直接搜索     | patternsearch       | 使用模式搜索查找函数最小值           |
| 遗传算法     | ga                  | 使用遗传算法寻找函数最小值           |
| 粒子群      | particleswarm       | 粒子群优化                   |
| 替代优化     | surrogateopt        | 用于以耗时最短为目标函数的替代优化       |
|          | packfcn             | 结合目标和非线性约束函数            |
| 模拟退火     | simulannealbnd      | 使用模拟退火算法查找函数最小值         |
| 多目标优化    | gamultiobj          | 使用遗传算法查找多个适应度函数的帕累托解的集合 |
|          | paretosearch        | 寻找帕累托集合中的点              |

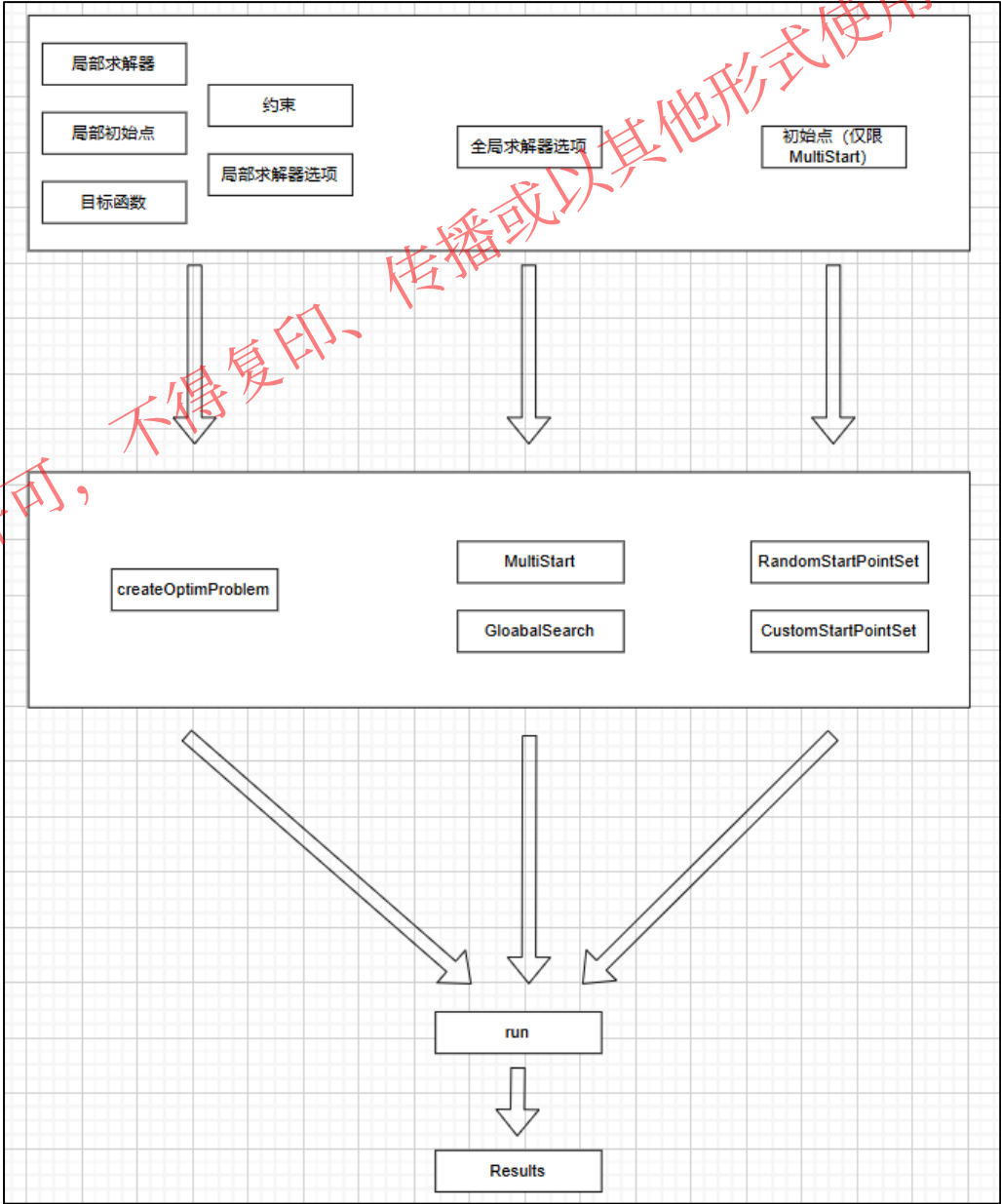


## 3.2 全局优化-全局或多起点搜索

这些求解器适用于具有平滑目标函数和约束的问题。它们反复运行优化工具箱求解器（如fmincon），试图找到一个全局解或多个局部解。

以下是GlobalSearch 和 MultiStart 的工作流（流程如右图）：

- 1.创建问题结构体
- 2.创建求解器对象
- 3.设置 MultiStart 的起点（可选，仅限 MultiStart）
- 4.运行求解器





## 3.2 全局优化-全局或多起点搜索

run的调用格式为:

```
x,fval,exitflag,output,solutions = run(gs,problem)
```

```
x,fval,exitflag,output,solutions = run(ms,problem,k)
```

```
x,fval,exitflag,output,solutions = run(ms,problem,startpts)
```

第一个参数指定GlobalSearch 求解器或MultiStart 求解器, 第二个参数指定优化问题。使用createOptimProblem 创建问题。k为起始点个数, startpts为MultiStart 指定起始点, 指定为CustomStartPointSet 对象、RandomStartPointSet 对象或此类对象的一维数组 (k与startpts仅限运行MultiStart启用) 。



## 3.2 全局优化-使用GlobalSearch搜索六驼峰问题全局最小值

例：求 $[x_1, x_2]$ ，使得六驼峰函数  $f(x_1, x_2) = (4x_1^2 - 2.1x_1^4 + \frac{1}{3}x_1^6 + x_1 * x_2 - 4x_2^2 + 4x_2^4)$  在约束条件  $-3 \leq x_1, x_2 \leq 3$  下取得最小值。

```
using TyGlobalOptimization
using TyOptimization
using TyRandom
gs = GlobalSearch(; rng=MT19937ar(5489))
function sixmin(x)
    return (4 * x[1]^2 - 2.1 * x[1]^4 + x[1]^6
/ 3 + x[1] * x[2] - 4 * x[2]^2 + 4 * x[2]^4)
end
problem = createOptimProblem(
    "fmincon";
    x0=[-1, 2],
    objective=sixmin,
    lb=[-3, -3],
    ub=[3, 3],
    options=optimoptions("fmincon";
Algorithm="sqp"),
)
x, fval = Base.run(gs, problem)
```

GlobalSearch stopped because it analyzed all the trial points.

All 29 local solver runs converged with a positive local solver exit flag.

```
x = [-0.08984200594212277,
0.7126563939610455]
fval = -1.0316284534898765
```

### 3.3 全局优化-用遗传算法求解非光滑函数

全局优化提供了遗传算法来解决光滑或非光滑优化问题，它是一种随机的、基于种群的算法，通过种群成员之间的变异和交叉进行随机搜索。具体来说，可以使用ga算法来求解如下函数的全局最优解：

$$\min z = func(x) , \quad s.t. \begin{cases} constraint\_ueq(x) \leq 0 \\ constraint\_eq(x) = 0 \\ lb \leq x \leq ub \end{cases}$$

ga函数的具体调用格式为：

```
x, fval, output = ga(func, lb, ub)
```

```
x, fval, output = ga(func, lb, ub, constraint_ueq, constraint_eq)
```

```
x, fval, output = ga(func, lb, ub, constraint_ueq, constraint_eq, options)
```

```
x, fval, output = ga(func, nvars, constraint_ueq, constraint_eq)
```

```
x, fval, output = ga(func, nvars, constraint_ueq, constraint_eq,options)
```

```
x, fval, output = ga(func, perm_size)
```

```
x, fval, output = ga(func, perm_size, options)
```



### 3.3 全局优化-用遗传算法求解非光滑函数

其中：func为目标函数，lb、ub为变量的边界约束，即 $lb \leq x \leq ub$ ，constraint\_ueq，constraint\_eq为分别表示不等式与等式约束元组，options为优化选项，使用ga\_options创建。x为返回的满足func取得全局最小值的点，fval是最优解对应的目标函数值，一般来说， $fval = func(x)$ 。返回值output有4个分量，包括迭代次数，函数评估次数，运行时间，停止状态。

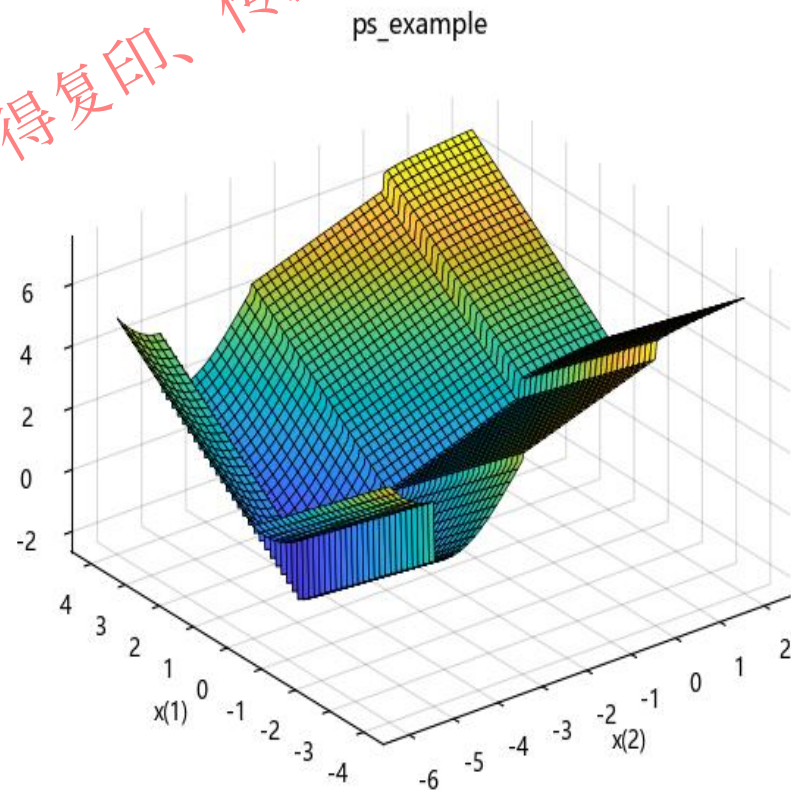
options = ga\_options() 用于为 ga 求解器创建默认选项。

options = ga\_options(iterations=1000)为 ga 设置选项，以使用最多 1000 次迭代。

### 3.3 全局优化-用遗传算法求解非光滑函数

运行以下代码以导入ps\_example函数，并绘制这个函数的图像。

```
using TyGlobalOptimization
using TyPlot
using TyBase
using PyCall
@pyimport numpy as np
xi = range(-6, 2, 300)
yi = range(-4, 4, 300)
X, Y = np.meshgrid(xi, yi)
A = [reshape(X, :, 1) reshape(Y, :, 1)]
Z = ps_example(A)
Z = reshape(Z, size(X))
surf(X, Y, Z)
title("ps_example")
ylabel("x(1)")
xlabel("x(2)")
```



### 3.3 全局优化-用遗传算法求解非光滑函数

优化案例：

```
#定义待求解问题
function func(p)
    x1, x2 = p
    return ps_example([x1 x2])[1]
end
lb = [-6.0, -4.0]
ub = [2.0, 4.0]

#执行求解
x, fval, output = ga(func, lb, ub)

#打印优化结果
println("最优解 = $x")
println("最优值 = $fval")
```

运行上述代码后，输出如下结果：

```
最优解 = [-4.716689741433464, -
0.0002139636118021898]
最优值 = -1.9997675398711094
```

### 3.4 全局优化-使用粒子群算法最小化带有非线性不等式约束的函数

粒子群可求解边界约束问题，其目标函数可以是非平滑的。如果对直接搜索（patternsearch）的优化效果不满意，可以尝试此方法。

其常用接口与遗传算法接口相似，具体地，调用格式有如下四种：

```
x, fval, output = particleswarm(func, lb, ub)
```

```
x, fval, output = particleswarm(func, lb, ub, constraint_ueq, constraint_eq)
```

```
x, fval, output = particleswarm(func, lb, ub, constraint_ueq, constraint_eq, options)
```

```
x, fval, output = particleswarm(problem)
```

$x$ ,  $fval$ ,  $output = particleswarm(func, lb, ub)$ ，用于求解带有边界约束的优化问题，其中  $func$  为目标函数， $lb$ ， $ub$  分别为决策变量  $x$  的上下界。 $x$ ,  $fval$ ,  $output = particleswarm(func, lb, ub, constraint\_ueq, constraint\_eq)$ ，用于求解带有边界约束及不等式和等式约束的优化问题，其中  $constraint\_ueq$ ， $constraint\_eq$  分别为不等式约束和等式约束。（如果不存在不等式或等式约束，令  $constraint\_ueq = ()$  或  $constraint\_eq = ()$ ）。 $options$ 为particleswarm求解器优化选项，使用  $pso\_options$  函数创建。 $problem$  为问题结构体，使用  $pso\_problem$  函数创建，可用于设置优化问题和求解器参数，特指包含属性  $obj$ 、 $lb$ 、 $ub$ 、 $constraint\_ueq$ 、 $constraint\_eq$ 、 $options$  的结构体。输出参数意义与  $ga$  函数相同，此处不做赘述。



### 3.4 全局优化-使用粒子群算法最小化带有非线性不等式约束的函数

例：求 $[x_1, x_2]$ ，使得函数  $f(x_1, x_2) = -20\exp(-0.2\sqrt{0.5(x_1^2 + x_2^2)}) - \exp(0.5(\cos(2\pi * x_1) + \cos(2\pi * x_2))) + 20 + e$  在约束条件  $(x_1 - 1)^2 + x_2^2 \leq 0.25, -6 \leq x_1 \leq 2, -4 \leq x_2 \leq 4$  下取得最小值。

#定义问题

```
using TyGlobalOptimization
```

```
function func(x)
```

```
    x1, x2 = x
```

```
    return -20 * exp(-0.2 * sqrt(0.5 * (x1^2 +  
x2^2))) - exp(0.5 * (cos(2 * pi * x1) + cos(2 *  
pi * x2))) + 20 + e
```

```
end
```

```
lb = [-6.0, -4.0]
```

```
ub = [2.0, 4.0]
```

```
cons_ueq1(x) = (x[1] - 1)^2 + (x[2] - 0)^2 -  
0.5^2
```

```
constraint_ueq = (cons_ueq1,)
```

#调用 particleswarm 优化该函数

```
x, fval, output = particleswarm(func, lb, ub,  
constraint_ueq, ())
```

#打印优化结果

```
println("最优解 = $x")
```

```
println("最优值 = $fval")
```

执行优化后打印结果如下：

最优解 = [0.952166547758943,

2.378608449489513e-9]

最优值 = 2.5799275570298694

检查不等式约束是否被满足：

```
println(cons_ueq1(x))
```

-0.24771196084670252

此时不等式约束在最优点处小于0，即表示满足约束。

## 3.5 全局优化-模拟退火优化

simulannealbnd 用于无导数无约束优化或边界优化的优化求解器，当其他（单目标）求解器无法满足您的需求时，请使用模拟退火。

其调用接口如下：

```
x, fval, output = simulannealbnd(func, x0, lb, ub)
```

```
x, fval, output = simulannealbnd(func, x0, lb, ub, options)
```

```
x, fval, output = simulannealbnd(problem)
```

其中，func为目标函数，x0为初始点，lb与ub分别为决策变量下限与上限，options为simulannealbnd 求解器的优化选项，使用sa\_options函数生成，problem为包含所有问题参数的结构体（包含属性：obj、x0、lb、ub、options），可由sa\_problem函数创建。

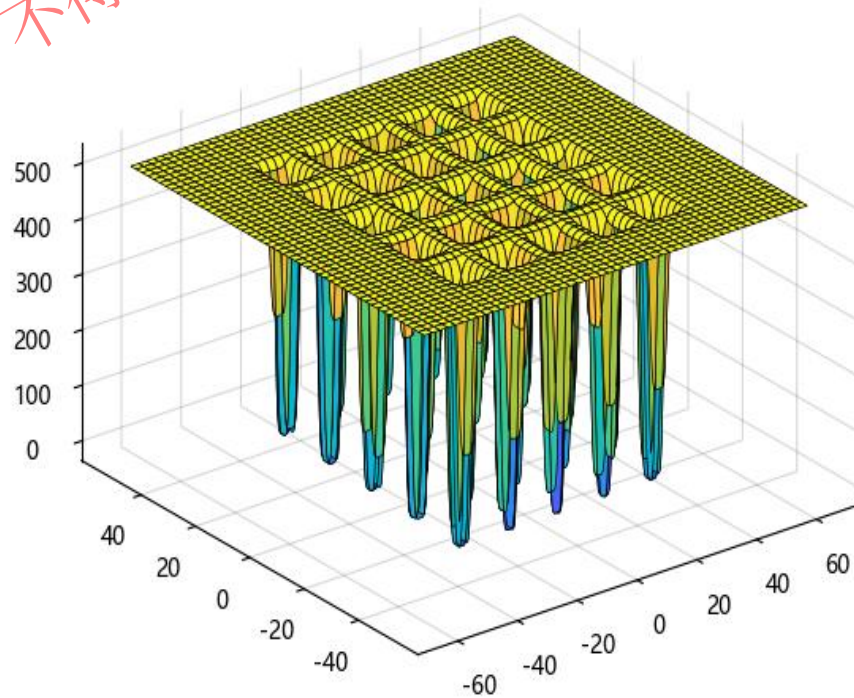
x为最优解，fval为最优解对应的目标函数值，一般来说， $fval = func(x)$ ，output 以 sa\_output 结构体形式存储，包括迭代次数，函数评估次数，运行时间，停止状态。



## 3.5 全局优化-模拟退火优化

例：在一个边界范围内，最小化 De Jong 的第五个函数。  
运行以下代码以导入dejong5fcn 函数，并绘制这个函数的图像。

```
using TyGlobalOptimization
using TyBase
using TyPlot
function func(x)
    res = dejong5fcn(x)
    return res
end
xi = range(-64, 64, 600)
yi = range(-54, 54, 600)
X, Y = meshgrid2(xi, yi)
A = [reshape(X, :, 1) reshape(Y, :, 1)]
Z = zeros(size(A, 1))
for i = 1:size(A, 1)
    Z[i] = func(A[i, :])
end
Z = reshape(Z, size(X))
surf(X, Y, Z)
```



## 3.5 全局优化-模拟退火优化

```
# 设定搜索起点和边界
x0 = [0.0, 0.0]
lb = [-64.0, -64.0]
ub = [64.0, 64.0]
# 执行求解
x, fval, output = simulannealbnd(func, x0, lb,
ub)
# 打印输出结果
println("最优解 = $x")
println("最优值 = $fval")
```

代码运行结果如下：

```
最优解 = [-31.97816502838519, -
31.97853332633746]
最优值 = 0.9980038377946738
```

## 3.6 全局优化-多目标优化

当您有多个想要同时优化的目标函数时，这些求解器会在相互竞争的目标函数之间找到最佳权衡结果。您可以使用gamultiobj 或 paretosearch 求解器来求解多目标问题，使用packfcn打包目标函数和等式/不等式约束（适用于 gamultiobj）。

gamultiobj为用遗传算法寻找多目标函数的帕累托前沿的求解器，其具体使用语法有以下三种：

```
PS, PF, output = gamultiobj(objcon, nobj, lb, ub)
```

```
PS, PF, output = gamultiobj(objcon, nobj, lb, ub, options)
```

```
PS, PF, output = gamultiobj(problem)
```

其中，objcon为优化对象函数，可调用 packfcn 打包生成，nobj 为目标个数，lb与ub分别为决策变量下限与上限，options为gamultiobj求解器的优化选项，使用gamultiobj\_options函数生成，problem为包含所有问题参数的结构体（包含属性：objcon、nobj、lb、ub、options），可由 gamultiobj\_problem 函数创建。

PS为帕累托解集，作为实矩阵返回，PF为帕累托前沿，output 以 gamultiobj\_output 结构体形式存储，包括迭代次数，函数评估次数，运行时间，停止状态。

## 3.6 全局优化-多目标优化

paretosearch用于寻找帕累托解集，其具体使用语法有以下三种：

```
res = paretosearch(obj, nvars, lb, ub)
```

```
res = paretosearch(obj, nvars, lb, ub, options)
```

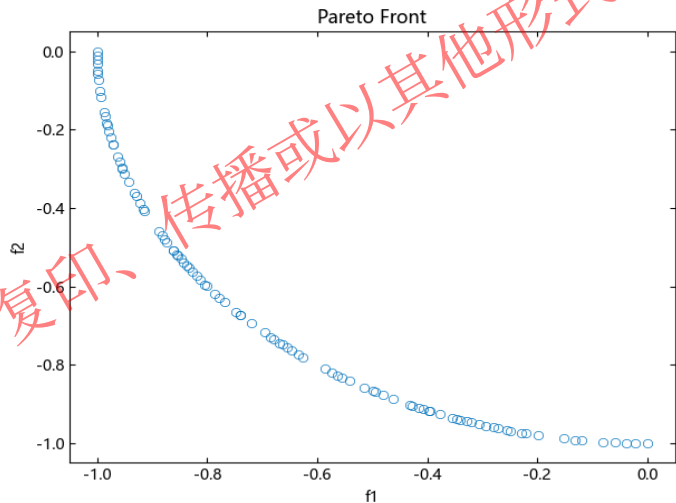
```
res = paretosearch(problem)
```

其中，obj为目标函数,可调用 packfcn 打包生成，nvars为决策变量维数，lb与ub分别为决策变量下限与上限，options为paretosearch求解器的优化选项，使用paretosearch\_options函数生成，problem为包含所有问题参数的结构体（包含属性：obj、nvars、lb、ub、options），可由 paretosearch\_problem 函数创建。res为输出结果的元组，包含帕累托解集ParetoSolution与帕累托前沿ParetoFront。

# 3.6 全局优化-多目标优化

例：寻找  $\sin(x)$  和  $\cos(x)$  在区间  $[0, 2\pi]$  上的帕累托前沿。

```
using TyGlobalOptimization
# 目标函数
f1(x) = sin(x[1])
f2(x) = cos(x[1])
f(x) = [f1(x), f2(x)]
# 目标个数
nobj = 2
# 不等式和等式约束
constraint_ueq = ()
constraint_eq = ()
objcon = packfcn(f, constraint_ueq, constraint_eq)
lb = zeros(1)      # 向量，元素个数代表决策变量维数
ub = ones(1)*2*pi
options = gamultiobj_options(p_m=0.1,
draw_picture=true) # 绘制帕累托前沿图
#执行求解
PS, PF, output = gamultiobj(objcon, nobj, lb, ub,
options)
```



Pareto Solution: [4.712389100843858  
3.14159343082972 ... 4.415615446153771  
4.257117424785442]  
Pareto Front: [-0.9999999999999928 -  
7.772399269630172e-7 ... -  
0.9562850007411857 -0.8981416566080743;  
1.204591677900659e-7 -  
0.999999999999996979 ... -0.2924363133357935  
-0.4397062254111306]  
其他输出信息: gamultiobj\_output(1000,  
199900, 1.754000186920166, "Maximum  
number of iterations exceeded.")

感谢您的聆听!

Thanks



未经授权，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究